SKILL – 14

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
   int fd;
   int saved_stdout;

   /* Save the original stdout */
   saved_stdout = dup(STDOUT_FILENO);

   if (saved_stdout == -1)
   {
      perror("dup");
      return 1;
   }

   /* Open the file in append mode */
   fd = open("append.txt",
         O_WRONLY | O_CREAT | O_APPEND,
         0644);

   if (fd == -1)
```

```
{
    perror("open append.txt");
    close(saved_stdout);
    return 1;
}

/* Redirect stdout to the file */
if (dup2(fd, STDOUT_FILENO) == -1)
{
    perror("dup2");
    close(fd);
    close(saved_stdout);
    return 1;
}

close(fd);

printf("New line appended to the file.\n");
printf("Existing content is preserved.\n");

/* Flush before restoring stdout */
if (fflush(stdout) == EOF)
{
    perror("fflush");
    dup2(saved_stdout, STDOUT_FILENO);
    close(saved_stdout);
    return 1;
}
```

```
    /* Restore original stdout */
    if (dup2(saved_stdout, STDOUT_FILENO) == -1)
    {
        perror("restore stdout");
        close(saved_stdout);
        return 1;
    }

    close(saved_stdout);

    printf("Append operation completed.\n");

    return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano append_redirect.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -o append_redirect app
end_redirect.c
snigdha@Snigdha:~/quotes_practical$ echo "Original content" > append.txt
cat append.txt
Original content
snigdha@Snigdha:~/quotes_practical$ ./append_redirect
Append operation completed.
snigdha@Snigdha:~/quotes_practical$ cat append.txt
Original content
New line appended to the file.
Existing content is preserved.
snigdha@Snigdha:~/quotes_practical$ ./append_redirect
cat append.txt
Append operation completed.
Original content
New line appended to the file.
Existing content is preserved.
New line appended to the file.
Existing content is preserved.
```

```
#include <stdio.h>
```

```
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

int main(void)
{
   int fd;
   int saved_stderr;

   /* Save original stderr */
   saved_stderr = dup(STDERR_FILENO);

   if (saved_stderr == -1)
   {
      perror("dup stderr");
      return 1;
   }

   /* Open/create the error log */
   fd = open("error.log",
         O_WRONLY | O_CREAT | O_TRUNC,
         0644);

   if (fd == -1)
   {
      perror("open error.log");
      close(saved_stderr);
      return 1;
```

```
}

/* Redirect stderr to error.log */
if (dup2(fd, STDERR_FILENO) == -1)
{
   perror("dup2 stderr");
   close(fd);
   close(saved_stderr);
   return 1;
}

close(fd);

/* Generate an error intentionally */
if (access("missing_file.txt", F_OK) == -1)
{
   perror("Error checking missing_file.txt");
}

fprintf(stderr, "This is a test error message.\n");

/* Restore original stderr */
if (dup2(saved_stderr, STDERR_FILENO) == -1)
{
   close(saved_stderr);
   return 1;
}
```

```
    close(saved_stderr);

    printf("Error messages were redirected to error.log.\n");

    return 0;
}
```

